

Министерство науки и высшего образования Российской Федерации

Санкт-Петербургский политехнический университет Петра Великого
Высшая школа программной инженерии

КУРСОВАЯ РАБОТА

ПО ДИСЦИПЛИНЕ «БАЗЫ ДАННЫХ»

ГРАФИЧЕСКИЙ ИНТЕРФЕЙС ПОЛЬЗОВАТЕЛЯ ДЛЯ БАЗЫ ДАННЫХ «ГОСПИТАЛЬ»

Выполнил
студент гр. 5130904/30103

Зайцев В.А.

Руководитель

Прокофьев О.В.

Санкт-Петербург

2025

Оглавление

1.	Задание	3
2.	Пользователи базы данных	3
3.	Работа интерфейса	3
4.	Вывод	3
5.	Список литературы	3

1. Задание
2. Пользователи базы данных
3. Работа интерфейса
4. Вывод
5. Список литературы

Ведомый контроллер SPI

Контроллер для приёма данных от внешних устройств по последовательному интерфейсу SPI. Контроллер принимает по шине SPI байт данных и передаёт его в управляемый модуль. После приёма сигнал valid находится в состоянии «1» в течение одного такта.

Входные сигналы:

```
bool  sclk, mosi, ss_n;
```

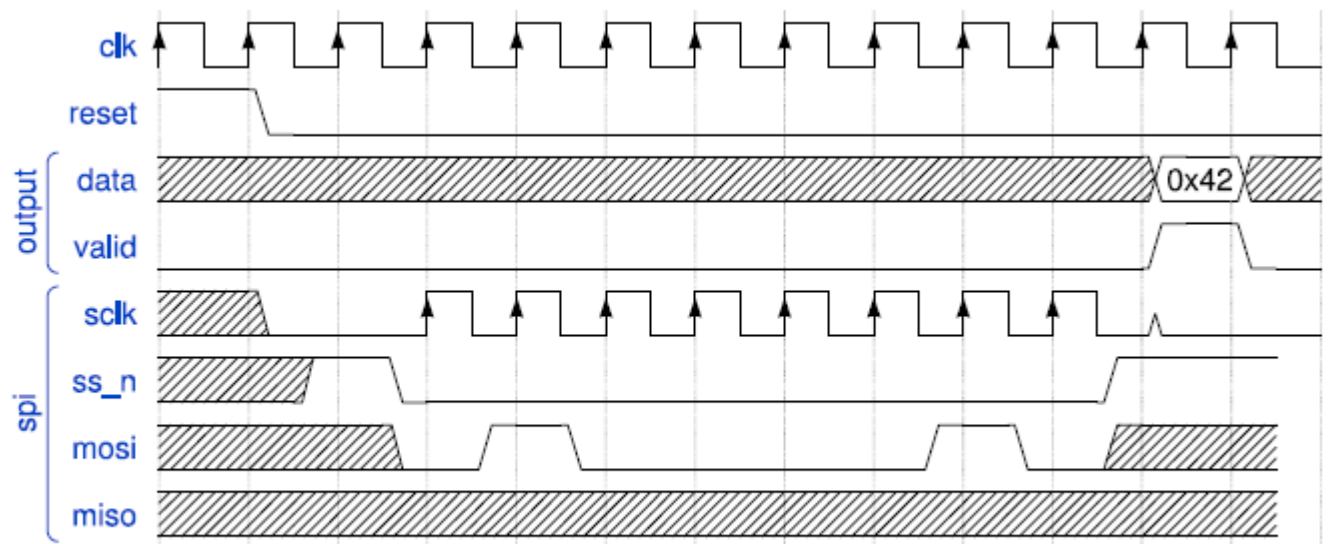
Выходные сигналы:

```
bool  miso;  
sc_uint<8> data;  
bool  valid;
```

Логика:

последовательная и комбинационная

Пример временной диаграммы:



Программная реализация на SystemC алгоритма Ведомый контроллер SPI

SPISlave.h

```
1  #ifndef SPISLAVE_H
2  #define SPISLAVE_H
3  #include <systemc.h>
4
5  SC_MODULE(SPISlave)
6  {
7      /*** INPUT SIGNALS: ***/
8      sc_in<bool> reset;
9      sc_in<bool> sclk;
10     sc_in<bool> mosi;
11     sc_in<bool> ss;
12     /*** OUTPUT SIGNALS: ***/
13     sc_out<bool> miso;
14     sc_out<bool> valid;
15     sc_out<sc_uint<8>> data;
16     /*** INTERNAL SIGNALS: ***/
17     sc_signal<bool> ss_int;
18     sc_signal<sc_uint<8> > buffer;
19     // methods
20     void set_outputs();
21     void update_buffer();
22     void update_ss();
23     // constructor of module
24     SC_CTOR(SPISlave)
25     {
26         SC_METHOD(update_buffer);
27         sensitive_pos << reset;
28         sensitive_neg << sclk;
29         SC_METHOD(set_outputs);
30         sensitive_pos << sclk << reset;
31         SC_METHOD(update_ss);
32         sensitive_pos << sclk;
33     }
34 };
35 #endif // SPISLAVE_H
36
```

SPISlave.cpp

```
1  #include "SPISlave.h"
2
3  void SPISlave::update_buffer()
4  {
5      if(reset)
6      {
7          buffer.write(0);
```

```

8      }
9      else
10     {
11         if (!ss.read())
12         {
13             buffer.write((sc_uint<8>(mosi.read()) << 7) | (buffer.read() >> 1));
14         }
15     }
16 }
17
18 void SPISlave::update_ss()
19 {
20     if (ss.read())
21     {
22         ss_int.write(1);
23     }
24     else
25     {
26         ss_int.write(0);
27     }
28 }
29
30 void SPISlave::set_outputs()
31 {
32     miso.write(buffer.read() & 1);
33     data.write(buffer.read());
34     if (reset)
35     {
36         valid.write(0);
37     }
38     else
39     {
40         if (!ss_int.read())
41         {
42             if (ss.read())
43             {
44                 valid.write(1);
45             }
46             else
47             {
48                 valid.write(0);
49             }
50         }
51         else
52         {
53             valid.write(0);
54         }
55     }

```

```
56     }
57
```

Test.h

```
1  #ifndef TEST_H
2  #define TEST_H
3  #include <iostream>
4  #include <systemc.h>
5
6  SC_MODULE(Test)
7  {
8      /** interface signals */
9      sc_in<bool> sclk;
10     sc_in<bool> valid;
11     sc_in<sc_uint<8> > data;
12     sc_out<bool> reset;
13     sc_out<bool> mosi;
14     sc_out<bool> ss;
15     // internal values
16     unsigned int counter;
17     unsigned char vals[10] = { 10, 29, 57, 255, 128, 185, 0, 1, 2, 3 };
18     // SC_THREAD: generates reset, ss and mosi signals
19     void generate_command();
20     // SC_METHOD: checks that data is same as expected
21     void check_data();
22     // constructor of test bench
23     SC_CTOR(Test):
24     counter(0)
25     {
26         SC_THREAD(generate_command);
27         sensitive_pos << sclk;
28         SC_METHOD(check_data);
29         dont_initialize();
30         sensitive_pos << valid;
31     }
32 };
33 #endif // TEST_H
34
```

Test.cpp

```
1  #include "Test.h"
2
3  void Test::generate_command()
4  {
5      reset.write(1);
6      ss.write(1);
7      wait(50, SC_NS);
8      reset.write(0);

```

```

9      wait(100, SC_NS);
10     while (counter < 10)
11     {
12         int tmp = vals[counter];
13         ss.write(0);
14         wait();
15         for (int i = 0; i < 8; ++i)
16         {
17             mosi.write(tmp & 1);
18             tmp >>= 1;
19             wait();
20         }
21         ss.write(1);
22         wait(200, SC_NS);
23     }
24 }
25 void Test::check_data()
26 {
27     if (vals[counter] != data.read())
28     {
29         std::cerr << "Error at" << sc_time_stamp() << ": SPI returned " << data
30 .read();
31         std::cerr << " when expected " << vals[counter] << std::endl;
32     }
33     ++counter;
34 }

```

main.cpp

```

1      #include <systemc.h>
2      #include "Test.h"
3      #include "SPISlave.h"
4
5      int sc_main(int argc, char* argv[])
6      {
7          // create modules
8          SPISlave module("SPI");
9          Test test("SPI_test");
10         //create 200MHz clock
11         sc_clock s_sclk("sclk", 5, SC_NS);
12         // create signals
13         sc_signal<bool> s_reset("reset");
14         sc_signal<bool> s_miso("miso");
15         sc_signal<bool> s_mosi("mosi");
16         sc_signal<sc_uint<8> > s_data("data");
17         sc_signal<bool> s_valid("valid");
18         sc_signal<bool> s_ss("ss");
19         // connect signals

```

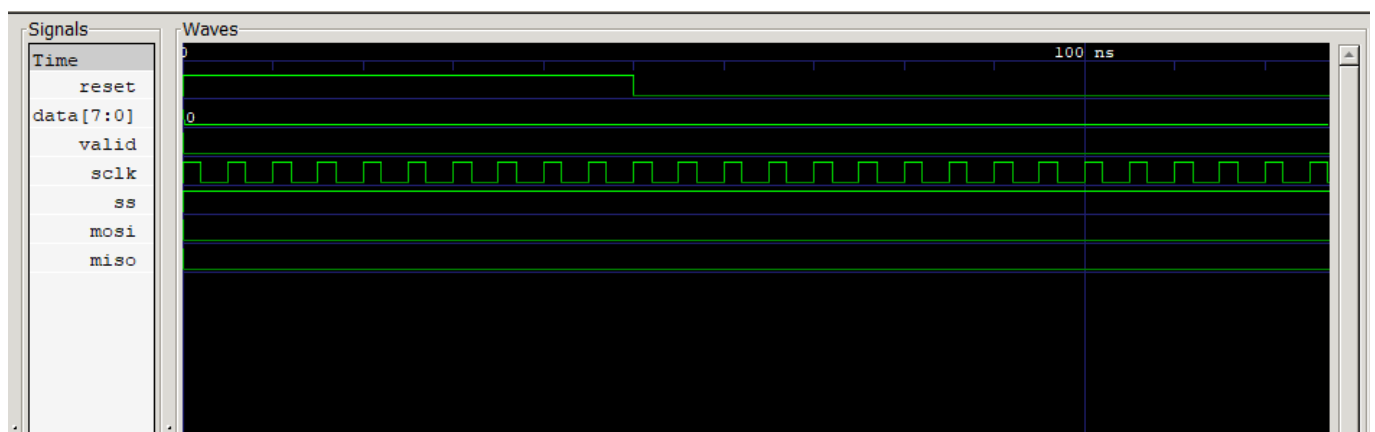


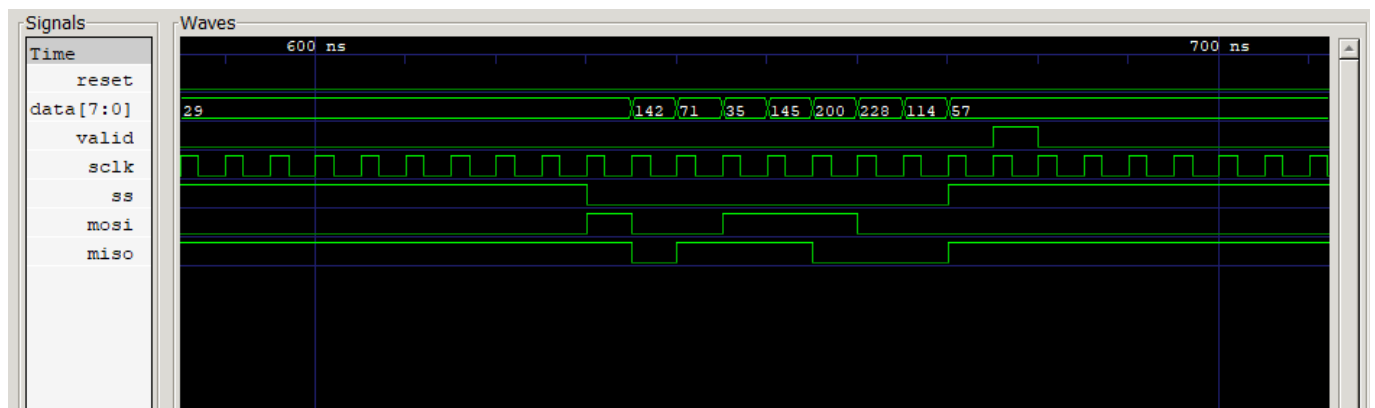
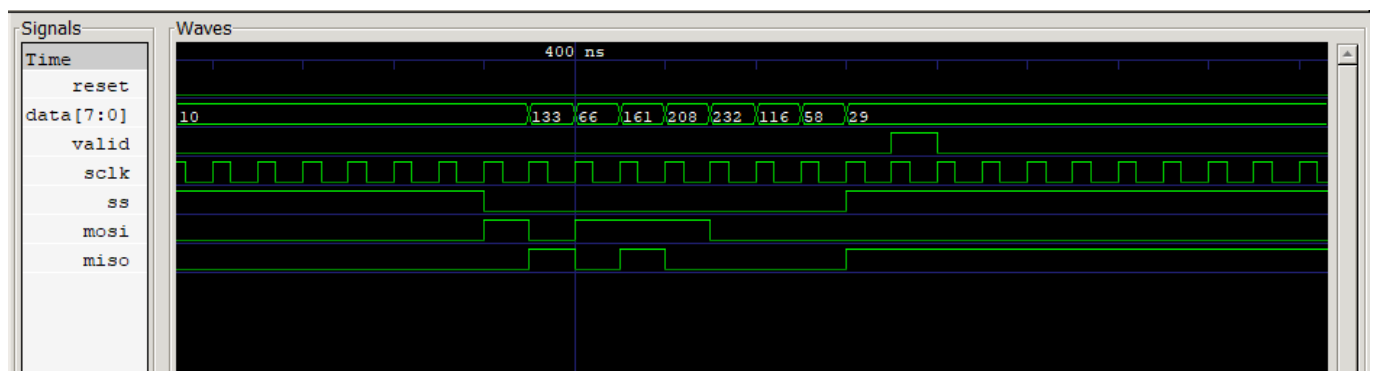
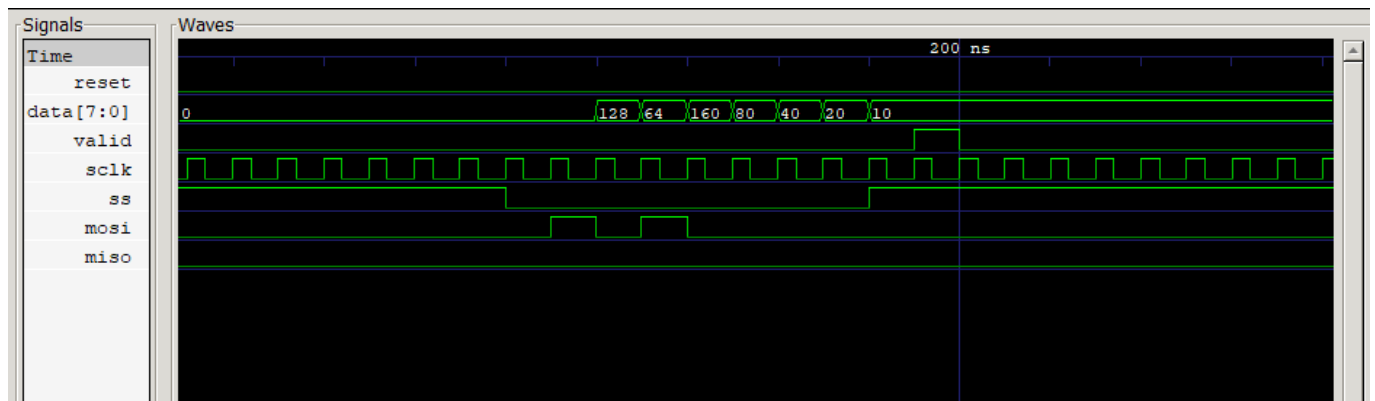
```

20     test.sclk(s_sclk);
21     test.reset(s_reset);
22     test.mosi(s_mosi);
23     test.ss(s_ss);
24     test.data(s_data);
25     test.valid(s_valid);
26     module.sclk(s_sclk);
27     module.reset(s_reset);
28     module.mosi(s_mosi);
29     module.miso(s_miso);
30     module.valid(s_valid);
31     module.data(s_data);
32     module.ss(s_ss);
33     // create output trace file
34     sc_trace_file* tf = sc_create_vcd_trace_file("SPI");
35     tf->set_time_unit(1, SC_NS);
36     sc_trace(tf, s_sclk, "sclk");
37     sc_trace(tf, s_reset, "reset");
38     sc_trace(tf, s_ss, "ss");
39     sc_trace(tf, s_mosi, "mosi");
40     sc_trace(tf, s_miso, "miso");
41     sc_trace(tf, s_valid, "valid");
42     sc_trace(tf, s_data, "data");
43     sc_start(10000, SC_NS);
44     sc_close_vcd_trace_file(tf);
45     return 0;
46 }
47

```

Тестирование симуляцией





Аппаратная реализация на Verilog HDL алгоритма Ведомый контроллер SPI

```

1  module SPISlave(data,valid,miso,ss,mosi,sclk,reset);
2  output [7:0] data;
3  output valid;
4  output miso;
5  input ss;
6  input mosi;
7  input sclk;
8  input reset;
9
10 reg [7:0] data;
```

```

11     reg valid;
12     reg miso;
13
14     reg [7:0] buffer;
15     reg ss_int;
16
17     //update_ss:
18     always @(posedge sclk )
19     begin
20         if (ss ) begin
21             ss_int  <=(1);
22         end
23         else begin
24             ss_int  <=(0);
25         end
26     end
27
28     //set_outputs:
29     always @(posedge reset or posedge sclk ) begin
30         miso  <=(buffer &1);
31         data  <=(buffer );
32         if (reset ) begin
33             valid  <=(0);
34         end
35         else begin
36             if (!ss_int ) begin
37                 if (ss ) begin
38                     valid  <=(1);
39                 end
40                 else begin
41                     valid  <=(0);
42                 end
43             end
44             else begin
45                 valid  <=(0);
46             end
47         end
48     end
49
50     //update_buffer:
51     always @(negedge sclk or posedge reset ) begin
52         if (reset ) begin
53             buffer  <=(0);
54         end
55         else begin
56             if (!ss ) begin
57                 buffer  <=((mosi )<<7)|(buffer >>1));
58             end

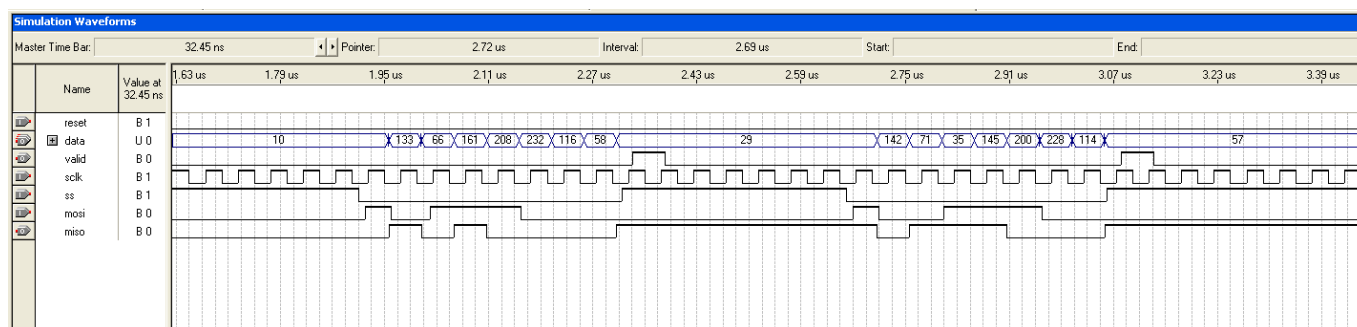
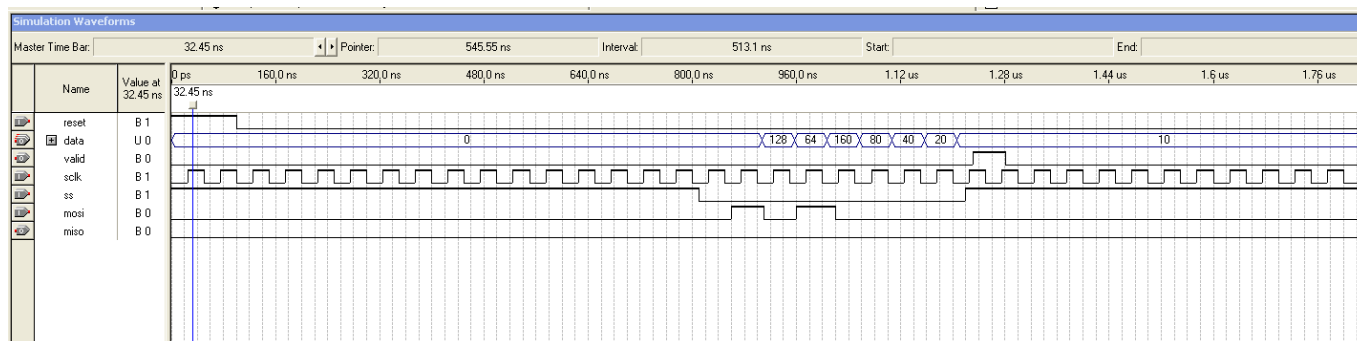
```

```

59     end
60     end
61     endmodule
62
63

```

Тестирование симуляцией

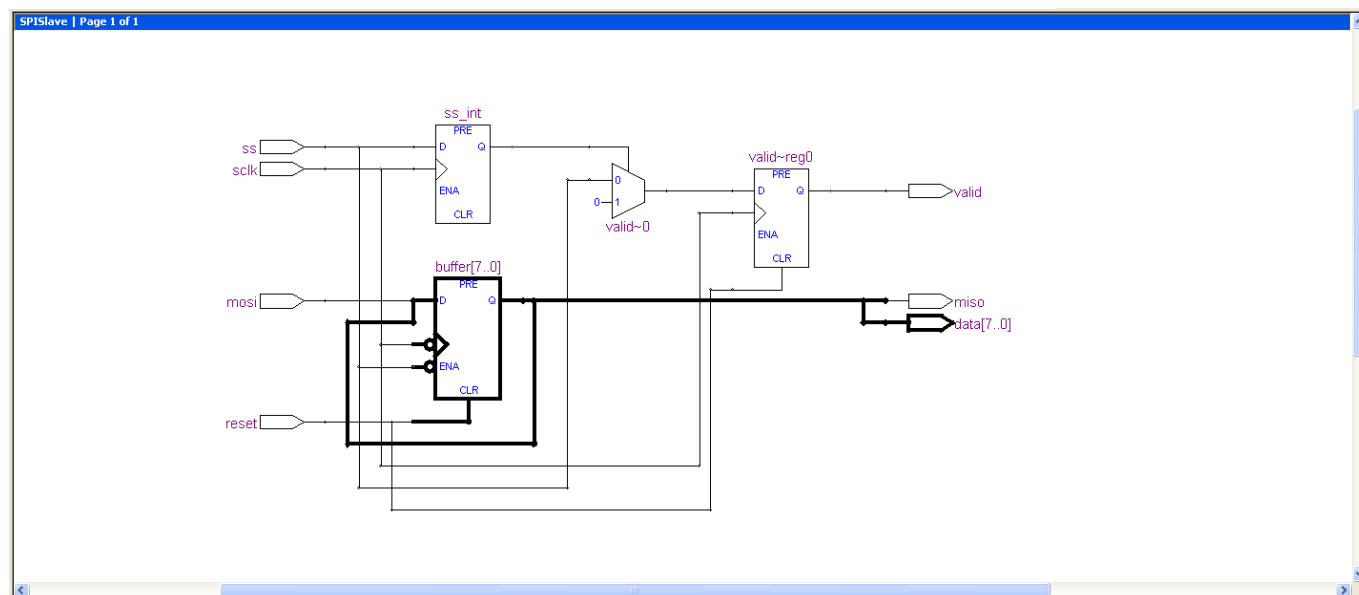


Проверка на синтезируемость

Отчёт в среде Quartus II

Flow Summary	
Flow Status	Successful - Wed Jun 02 15:55:47 2021
Quartus II Version	5.0 Build 148 04/26/2005 SJ Full Version
Revision Name	SPI
Top-level Entity Name	SPISlave
Family	Stratix
Met timing requirements	Yes
Total logic elements	10 / 10,570 (< 1 %)
Total pins	14 / 336 (4 %)
Total virtual pins	0
Total memory bits	0 / 920,448 (0 %)
DSP block 9-bit elements	0 / 48 (0 %)
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 2 (0 %)
Device	EP1S10F494C5
Timing Models	Final

RTL схема



Технологическая схема

